



STACK and QUEUE

Nazirul Hasan
Lecturer, CSE, KUET



Introduction to Stacks

Stack ADT



Introduction to Stacks

Stack ADT

↳ features/Operations ✓

implementation ✗



Introduction to Stacks

Stack ADT



Last-In-First-Out (LIFO)



Introduction to Stacks

Stack ADT

A list with the restriction
that insertion and deletion
can be performed only from
one end, called the top.



Introduction to Stacks

Stack ADT

A list with the restriction that insertion and deletion can be performed only from one end, called the top.

Operations

- (1) Push(x)
- (2) Pop()
- (3) Top()
- (4) IsEmpty()



Introduction to Stacks

Stack ADT

A list with the restriction that insertion and deletion can be performed only from one end, called the top.

Operations

(1) Push(x)

(2) Pop()

(3) Top()

(4) IsEmpty()

} Constant
time
or
 $O(1)$



Introduction to Stacks

Stack ADT (LIFO)

Operations

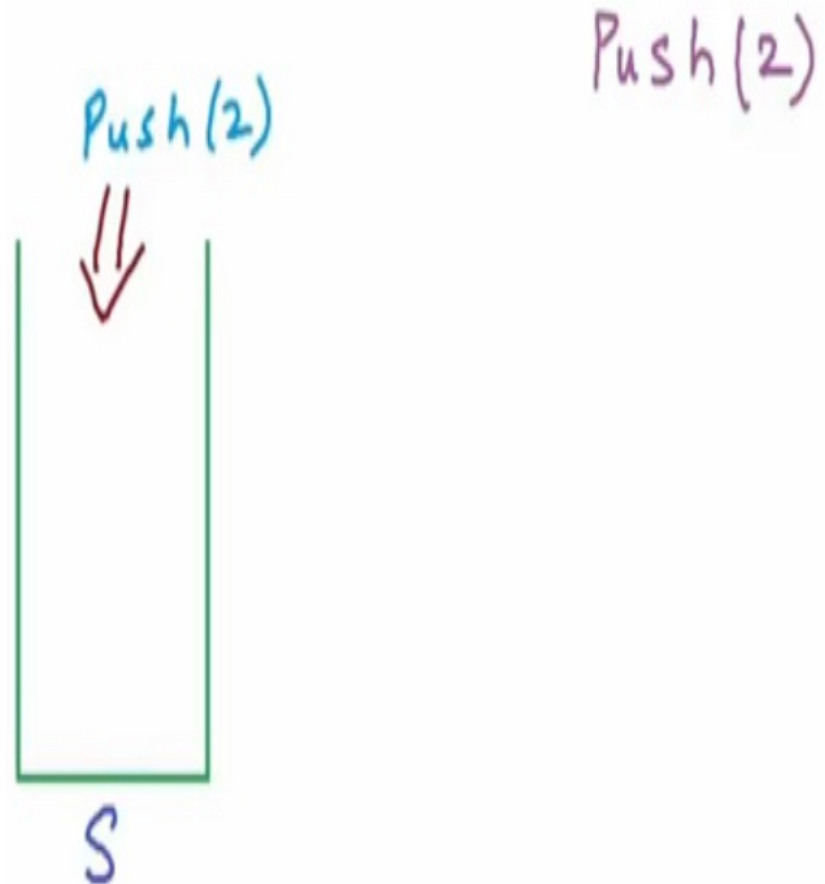
- (1) Push(x)
- (2) Pop()
- (3) Top()
- (4) IsEmpty()





Introduction to Stacks

Stack ADT (LIFO)



Operations

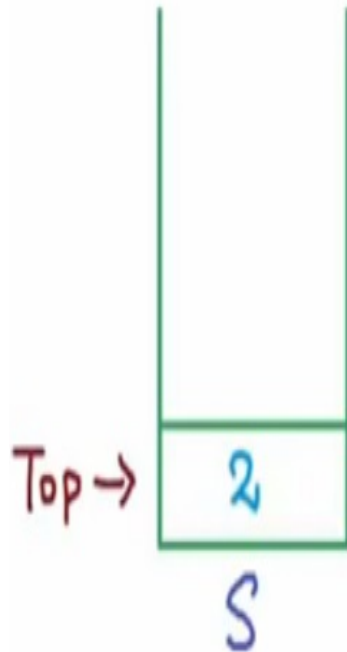
- (1) Push(x)
- (2) Pop()
- (3) Top()
- (4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)

Push(2)



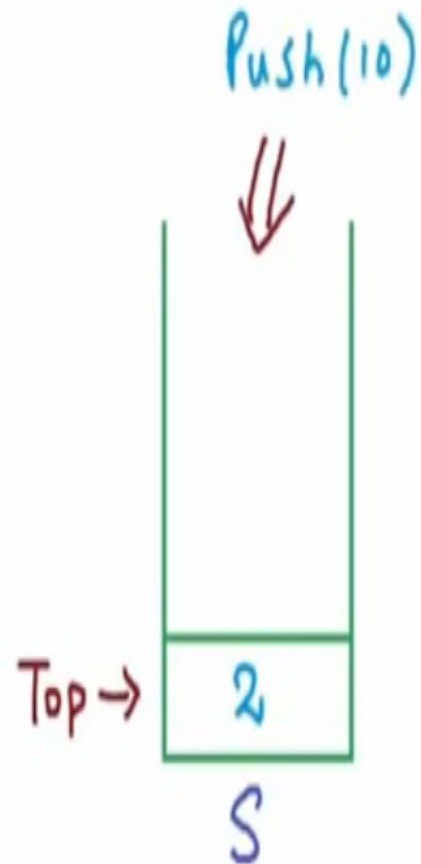
Operations

- (1) Push(x)
- (2) Pop()
- (3) Top()
- (4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Operations

(1) Push(x)

(2) Pop()

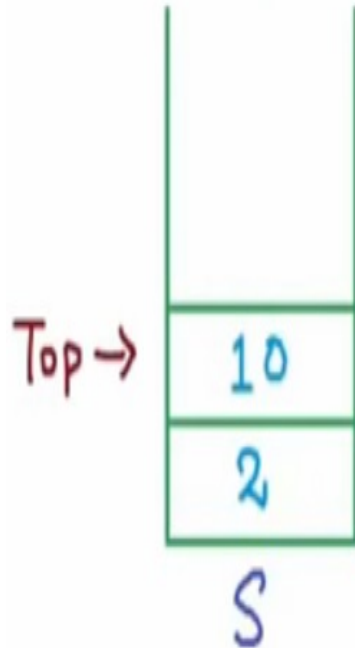
(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Operations

(1) Push(x)

(2) Pop()

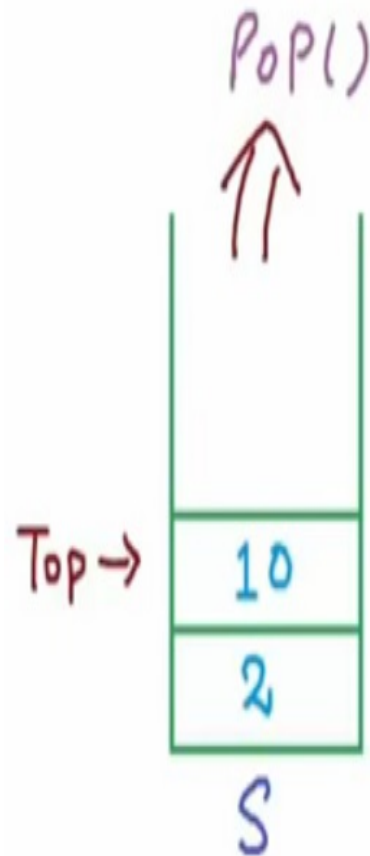
(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Pop()

Operations

(1) Push(x)

(2) Pop()

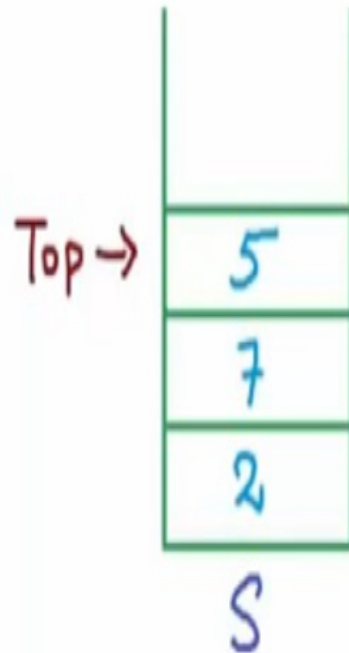
(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Pop()

Push(7)

Push(5)

Operations

(1) Push(x)

(2) Pop()

(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Pop()

Push(7)

Push(5)

Top() $\Rightarrow$ 5

Operations

(1) Push(x)

(2) Pop()

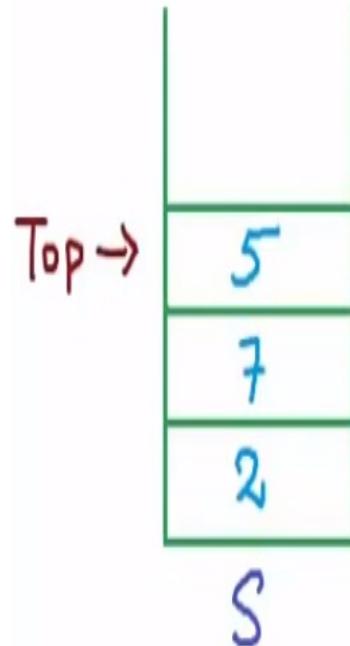
(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Pop()

Push(7)

Push(5)

Top() ⇒ 5

IsEmpty() ⇒ false

Operations

(1) Push(x)

(2) Pop()

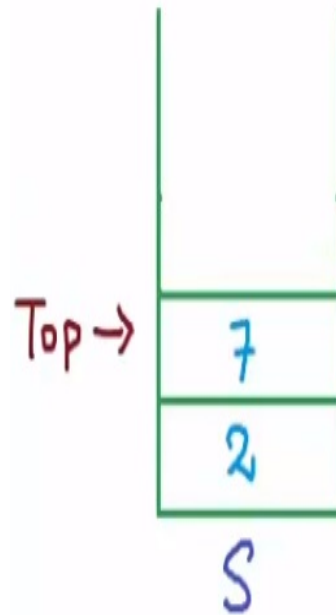
(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Pop()

Push(7)

Push(5)

Top() $\Rightarrow$ 5

IsEmpty() $\Rightarrow$ false

Pop()

Operations

(1) Push(x)

(2) Pop()

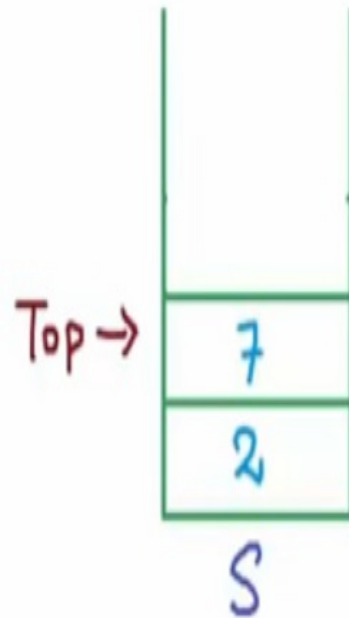
(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Pop()

Push(7)

Push(5)

Top() $\Rightarrow$ 5

IsEmpty() $\Rightarrow$ false

Pop()

Pop()

Operations

(1) Push(x)

(2) Pop()

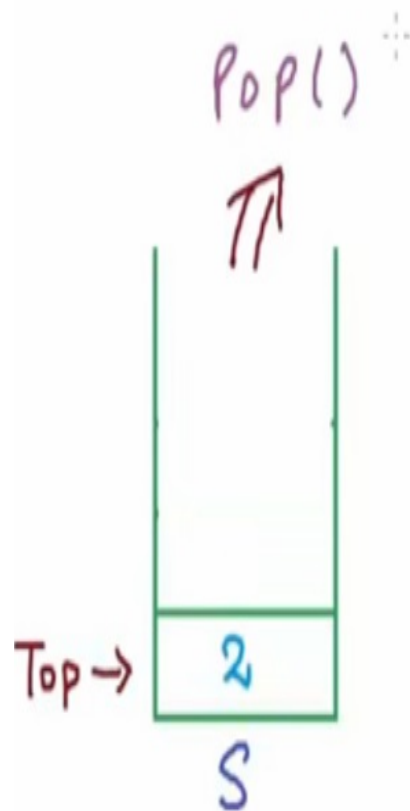
(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)
Push(10)
Pop()
Push(7)
Push(5)
Top() $\Rightarrow$ 5
IsEmpty() $\Rightarrow$ false
Pop()
Pop()
Pop()

Operations

- (1) Push(x)
- (2) Pop()
- (3) Top()
- (4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)



Push(2)

Push(10)

Pop()

Push(7)

Push(5)

Top() $\Rightarrow$ 5

IsEmpty() $\Rightarrow$ false

Pop()

Pop()

Pop()

Operations

(1) Push(x)

(2) Pop()

(3) Top()

(4) IsEmpty()



Introduction to Stacks

Stack ADT (LIFO)

Applications

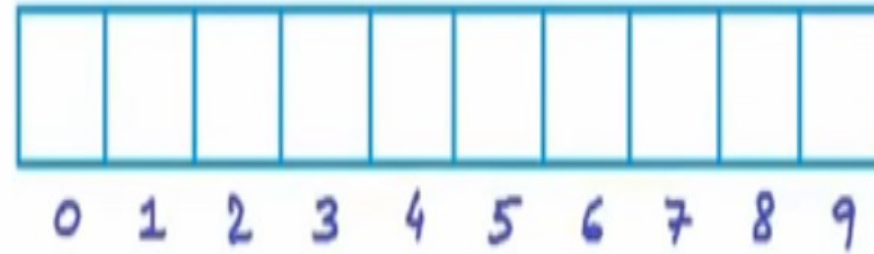
- Function Calls / Recursion
- Undo in an editor
- Balanced Parentheses



Implementation of stacks

Array implementation

```
int A[10]
```

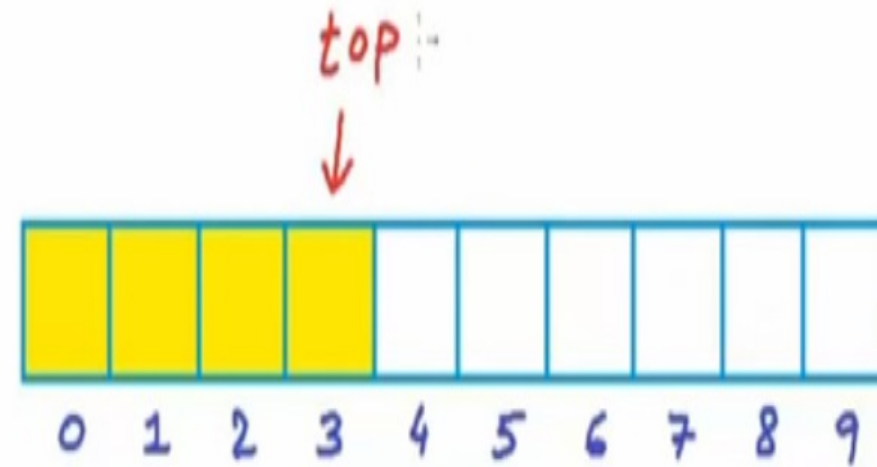




Implementation of stacks

Array implementation

```
int A[10]
```





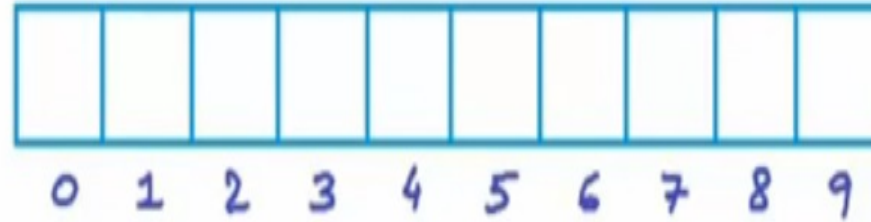
Implementation of stacks

Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

top
↓





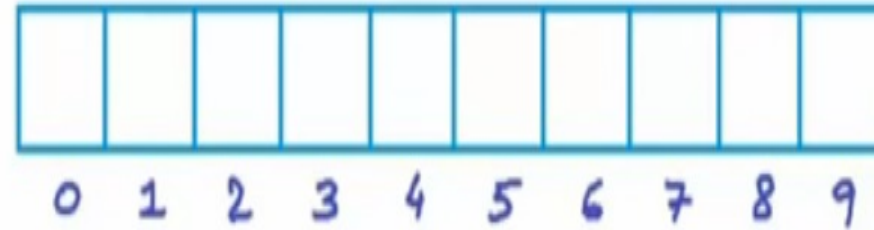
Implementation of stacks

Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

top
↓



```
Push(x)
```

```
{
```

```
    top ← top + 1
```

```
    A[top] ← x
```

```
}
```



Implementation of stacks

Array implementation

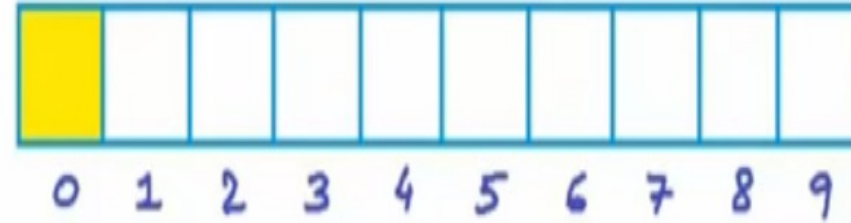
```
int A[10]
```

```
top ← -1 //empty stack
```

```
Push(x)
```

```
{  
⇒ top ← top + 1  
  A[top] ← x  
}
```

top
↓



Push(2)



Implementation of stacks

Array implementation

```
int A[10]
```

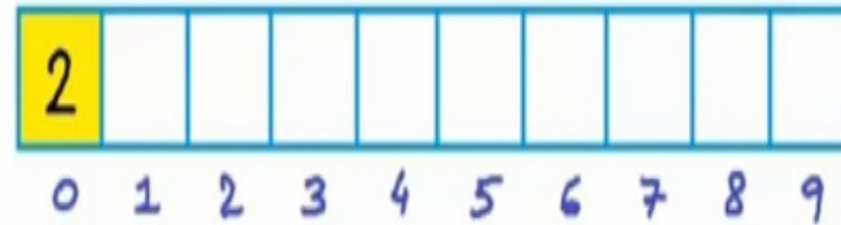
```
top ← -1 //empty stack
```

```
Push(x)
```

```
{  
    top ← top + 1
```

```
⇒ A[top] ← x  
}
```

top
↓



Push(2)



Implementation of stacks

Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

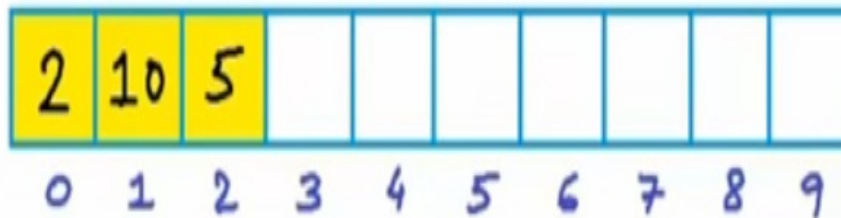
```
Push(x)
```

```
{
```

```
    top ← top + 1
```

```
⇒ A[top] ← x  
}
```

top



Push(2)

Push(10)

Push(5)



Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

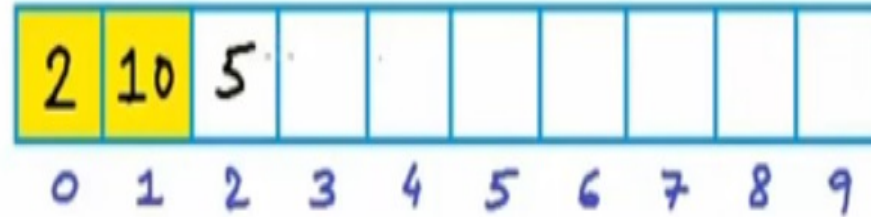
```
Push(x)
```

```
{  
    top ← top + 1  
    A[top] ← x  
}
```

```
Pop()
```

```
{  
    top ← top - 1  
}
```

top
↓



Push(2)

Push(10)

Push(5)

Pop()



Array implementation

```
int A[10]
```

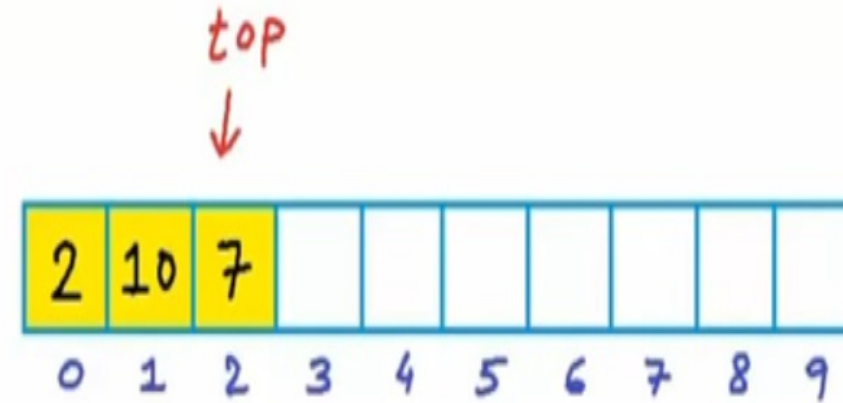
```
top ← -1 //empty stack
```

```
Push(x)
```

```
{  
    top ← top + 1  
    A[top] ← x  
}
```

```
Pop()
```

```
{  
    top ← top - 1  
}
```



```
Push(2)
```

```
Push(10)
```

```
Push(5)
```

```
Pop()
```

```
Push(7)
```



Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

```
Push(x)
```

```
{  
  top ← top + 1
```

```
  A[top] ← x
```

```
}
```

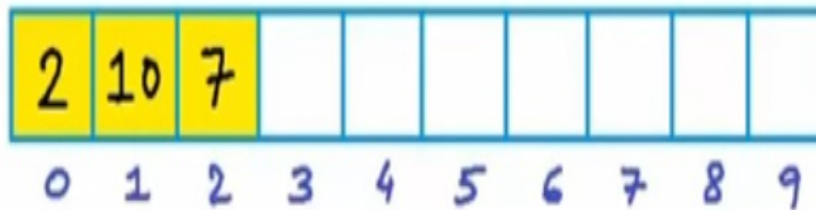
```
Pop()
```

```
{
```

```
  top ← top - 1
```

```
}
```

top
↓



→ Constant time
or

→ $O(1)$

Push(2)

Push(10)

Push(5)

Pop()

Push(7)



Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

```
Push(x)
```

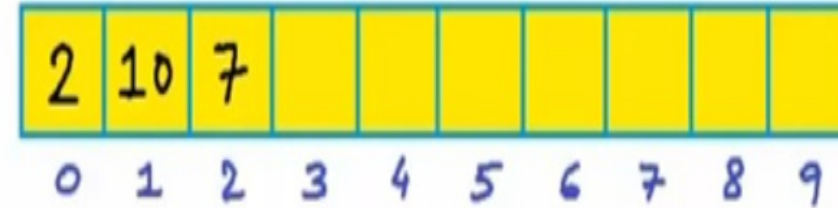
```
{  
  top ← top + 1  
  A[top] ← x  
}
```

→ Constant time
or

→ $O(1)$

```
Pop()
```

```
{  
  top ← top - 1  
}
```



Push(2)

Push(10)

Push(5)

Pop()

Push(7)



Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

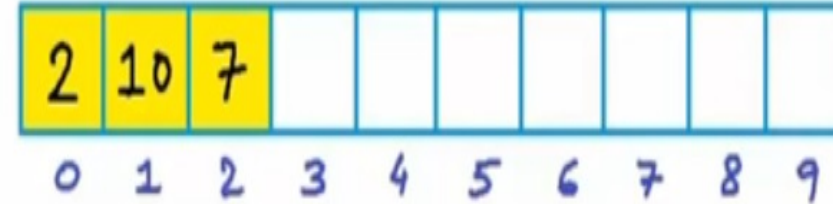
```
Push(x)
```

```
{  
  top ← top + 1  
  A[top] ← x  
}
```

```
Pop()
```

```
{  
  top ← top - 1  
}
```

top
↓



Overflow

↳ create a larger array. copy all elements in new array.

Cost - $O(n)$

where n = no. of elements in stack

Push(2)

Push(10)

Push(5)

Pop()

Push(7)



Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

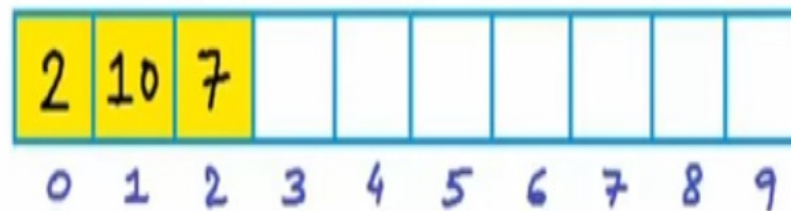
```
Push(x)
```

```
{  
  top ← top + 1  
  A[top] ← x  
}
```

```
Pop()
```

```
{  
  top ← top - 1  
}
```

top
↓



```
Top()
```

```
{  
  return A[top]  
}
```

```
IsEmpty()
```

```
{  
  if (top == -1)  
    return true  
  else  
    return false  
}
```

```
Push(2)
```

```
Push(10)
```

```
Push(5)
```

```
Pop()
```

```
Push(7)
```




QUEUE



- ☐ Who is at front gets the priority to enter first
- ☐ New people stands at the rare
- ☐ If no one enters, the bus is empty

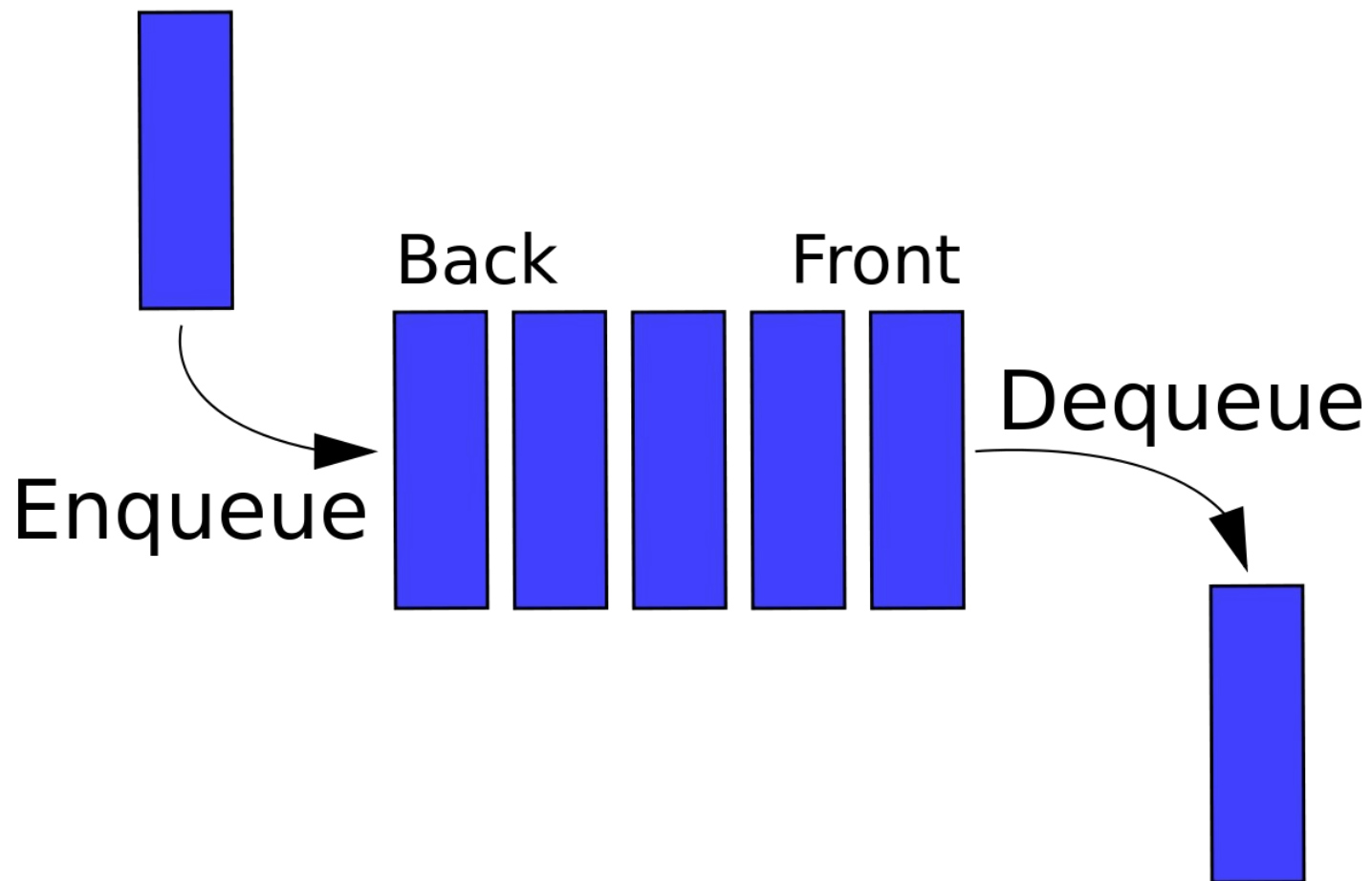


WHAT IS QUEUE?

- ABSTRACT DATA TYPE
- linear data structure
- FIFO – First in first out
- flexible with size
- Constant time ' $O(1)$ ' for each operation



QUEUE



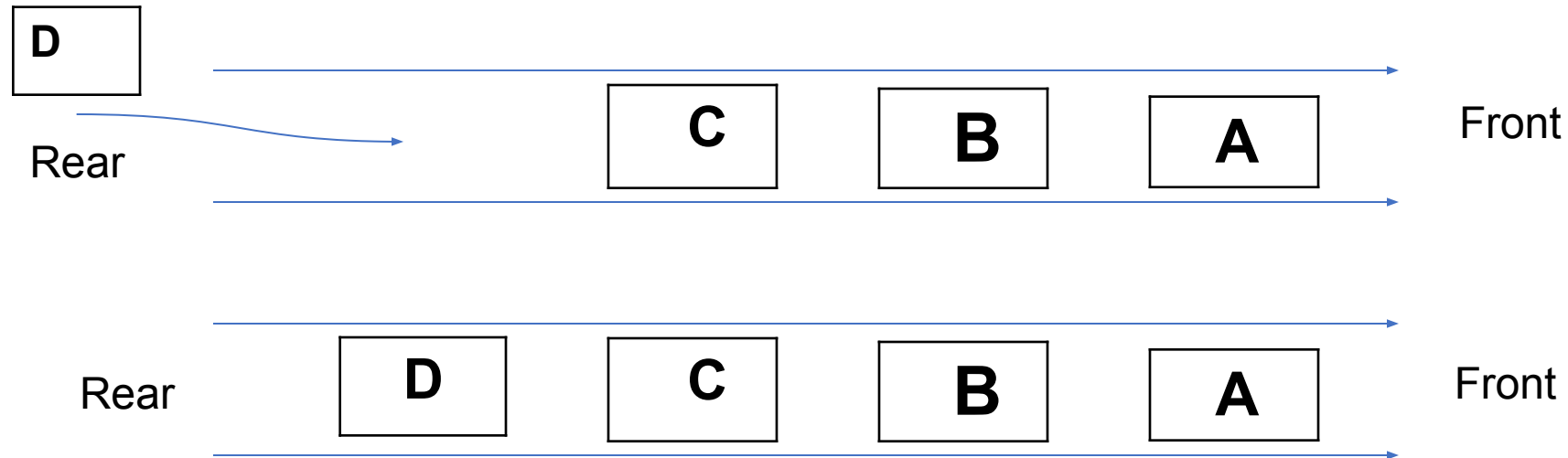


Operations in a QUEUE

- Enqueue/Insert/Push –at tail or rear
- Dequeue/Remove/Pop –at front or head
- Check the Front/Peak value
- Is our queue empty ?



Enqueue or Push

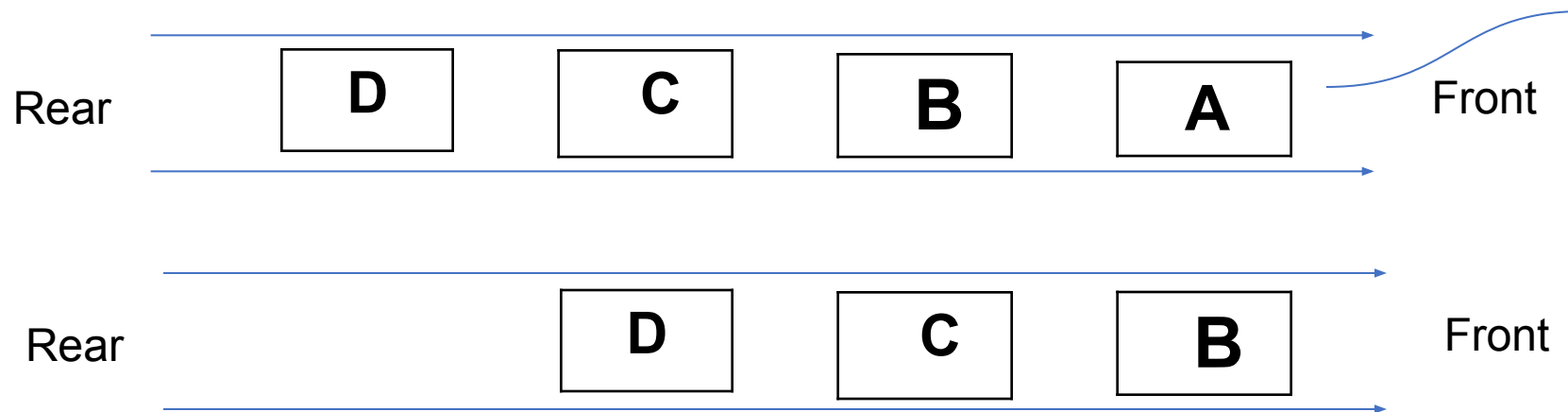


Enqueue in a queue

- Assume our function is **void enqueue(value)**
- We should pass items to be inserted as parameter
- Function should return nothing



Dequeue or Pop



Dequeue in a queue

- Assume our function is **int dequeue()**
- No parameter is needed
- Should return the value which is removed/popped



Check Front & Empty or not

- Assume our functions are **int front()** & **bool isEmpty()**
- **Front()** should return the value at the front or head
- **isEmpty()** should return whether the queue is empty or not
- Both functions need no parameter



QUEUE

- Array implementation
- Linked list implementation
- Both implementation need constant time $O(1)$ for each of the operations
- Has built in functions in C++ STL



DEQUE

- ❖ Deque is a Double Ended Queue
- ❖ Allows elements to be added or removed from both ends





OPERATIONS OF DEQUE

Basic Four operations of DEQUE

- ❖ Insert element at Back
- ❖ Delete element from Back
- ❖ Insert element at Front
- ❖ Delete element from Front



INSERT BACK



↑
Front

↑
Back

Push (55) at the Back
of the DEQUE



INSERT BACK



↑
Back
↑
Front

Push (7) at the Back
of the DEQUE



ALGORITHM FOR INSERTING AT BACK

- ❖ Step1. Start
- ❖ Step2. Check the queue is full or not as if $(r == \text{max}-1) \ \&\& \ (f == 0)$ if yes queue is full
- ❖ Step3. If false update the pointer r as $r = r + 1$
- ❖ Step4. Insert the element at pointer r as $Q[r] = \text{element}$
- ❖ Step5. Stop



INSERT FRONT



Front



Back

Push (4) at the front of
the DEQUE



INSERT FRONT



↑
Back

↑
Front

Push (-5) at the front
of the DEQUE



ALGORITHM FOR INSERTING AT FRONT

- Step1. Start
- Step2. Check the stack is full or not as if $(r == \text{max}-1) \ \&\& \ (f == 0)$
- Step3. If false update the pointer f as $f = f-1$
- Step4. Insert the element at pointer f as $Q[f] = \text{element}$
- Step5. Stop



REMOVE FRONT



↑
Back

↑
Front

Remove (-5) at the front of the DEQUEUE



ALGORITHM FOR REMOVING FROM FRONT

- ❖ Step1. Start
- ❖ Step2. Check the queue is empty or not as if $(f==r) \ \&\& \ (f==0)$ if yes queue is empty
- ❖ Step3. If false update the pointer f as $f=f+1$ and delete element at the position f as $\text{element} = Q[f]$
- ❖ Step4. If $(f==r)$ reset pointer f and r as $f=r=-1$
- ❖ Step5. Stop



REMOVE BACK



↑
Rear

↑
Front

Remove (7) at the
Back of the DEQUE



ALGORITHM FOR REMOVING FROM BACK

- ❖ Step1. Start
- ❖ Step2. Check the queue is empty or not as if $(f==r)$ if yes queue is empty
- ❖ Step3. If false update the pointer f as $r=r-1$ and delete element at the position f as $\text{element} = Q[f]$
- ❖ Step4. If $(f==r)$ reset pointer f and r as $f=r=-1$
- ❖ Step5. Stop



COMPLEXITY

The complexity of common operations on DEQUE

- ❖ Random Access – Constant time or **$O(1)$**
- ❖ Insertion or Removal of elements at the beginning or end
- Constant time **$O(1)$**



REFERENCES

- Schaum's Outlines Data Structures with C by Seymour Lipschutz.
- <https://www.geeksforgeeks.org/stack-data-structure/>
- https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.htm
- <https://www.geeksforgeeks.org/queue-data-structure/>
- <https://www.scaler.com/topics/dequeue-in-data-structure/>